

SMARTBAND V001

Hardware Integration & System Demonstration Report

Task 3 — Peripheral Integration Phase

Hochschule Hamm-Lippstadt (HSHL)

Student: Masrur

Date: June 2026

Repository: Band01_Rev1 (Private GitHub)

1. Overview

This report documents the hardware integration phase of the Smartband V001 project, conducted on the Raspberry Pi 5 platform. The objective of this phase was to establish and verify a complete end-to-end health monitoring pipeline, integrating Bluetooth communication, OLED display output, LED indicators, buzzer alerts, real-time data visualization, and cloud telemetry via ThingsBoard.

All hardware components were connected, tested individually, and then integrated into a unified Python-based system. Simulated sensor data (Heart Rate, Temperature, Acceleration) was used as a placeholder until the actual Band01_Rev1 Smartband hardware is assembled.

2. Hardware Components

The following hardware components were integrated during this phase:

Component	Model / Type	Interface	GPIO / Address
Raspberry Pi	Raspberry Pi 5	Host platform	—
Bluetooth Module	Pmod BT2 (RN-42)	UART	GPIO 14 (TX), GPIO 15 (RX)
OLED Display	SSD1306 128x64	I2C	0x3C (SDA: GPIO2, SCL: GPIO3)
RGB LED	Common Cathode RGB LED	GPIO	Red: GPIO27, Green: GPIO26
Buzzer	Active Buzzer Module	GPIO	GPIO17
GPIO Extender	T-Shaped Extension Board	Ribbon Cable	40-pin breakout

3. Wiring & Pin Mapping

3.1 OLED Display (SSD1306)

OLED Pin	Extension Board Label	Breadboard Row
VCC	3V3 (left side)	e5
GND	GND (left side)	e9
SDA	SDA1 (left side)	e6
SCL	SCL1 (left side)	e7

3.2 Pmod BT2 Bluetooth Module

Pmod J1 Pin	Signal	Extension Board Label	Breadboard
Pin 2	RX	TXD0 (right side)	f8

Pmod J1 Pin	Signal	Extension Board Label	Breadboard
Pin 3	TX	RXD0 (right side)	f9
Pin 5	GND	GND (right side)	i21
Pin 6	VCC	3V3 (left side)	e13

3.3 RGB LED & Buzzer

Component	Signal	GPIO	Extension Label	Breadboard
RGB LED	Red (+)	GPIO27	GPIO27	a40 via 330Ω resistor
RGB LED	Green (+)	GPIO26	GPIO26	a42 via 330Ω resistor
RGB LED	GND (-)	GND	GND	a41
Buzzer	(+)	GPIO17	GPIO17	a30
Buzzer	(-)	GND	GND	a34

4. Software Architecture

4.1 System Files

File	Purpose
smartband_full_v3.py	Main system — sensor simulation, OLED, LED, buzzer, BT, ThingsBoard, CSV
smartband_plot.py	Real-time matplotlib dashboard — reads sensor_data.csv and plots live graphs
font5x8.bin	Bitmap font required by adafruit_ssd1306 for OLED text rendering

4.2 Data Pipeline

The complete data flow for the system is as follows:

- Sensor simulator generates HR, Temperature, and Acceleration values every 2 seconds
- Values are written to sensor_data.csv for live graph consumption
- Values are sent to ThingsBoard cloud via MQTT (paho-mqtt)
- Values are transmitted via UART directly to the Pmod BT2 (/dev/serial0) for wireless Bluetooth delivery to a connected Android device
- OLED display shows live sensor readings, alternating with date/time and system status
- LED and buzzer respond to emergency threshold conditions

4.3 Emergency Logic

The system evaluates three threshold conditions on every cycle:

Parameter	Normal Range	Emergency Threshold	Response
Heart Rate	< 100 BPM	> 100 BPM	Red LED flash, buzzer ON, OLED emergency screen
Temperature	< 38.5°C	> 38.5°C	Red LED flash, buzzer ON, OLED emergency screen
Acceleration	< 2.0 G	> 2.0 G	Red LED flash, buzzer ON, OLED emergency screen

On a rising edge (normal → emergency), the buzzer activates and the LED flashes red. On a falling edge (emergency → normal), the system resets and the LED returns to green. Every 8 simulation cycles, a spike is deliberately injected to trigger and demonstrate the emergency response.

5. Bluetooth Integration

5.1 Pmod BT2 Module Details

Property	Value
Chip	Microchip RN-42
Device Name	RNBT-0793
Bluetooth Address (MAC)	00:06:66:F9:07:93
Baud Rate	115200 bps, 8N1
Bluetooth Profile	Classic Bluetooth SPP (Serial Port Profile)
Mode	Slave (waits for incoming connections)
Authentication	Disabled (SA,0) for open pairing
UART Device on Pi	/dev/serial0

5.2 Connection Method

In the final v3 implementation, data is transmitted directly from the Raspberry Pi via UART (/dev/serial0) to the Pmod BT2 module, which then wirelessly broadcasts it to any connected Bluetooth SPP device. This approach eliminates the need for rfcmm and allows external devices (Android phones, laptops) to connect directly to RNBT-0793.

5.3 Android Verification

The Bluetooth data stream was successfully verified using the Serial Bluetooth Terminal application on an Android device. The device paired with RNBT-0793 and received live sensor data in the following format:

Data Type	Format Example
Normal reading	HR:75 T:36.6 A:1.2
Emergency reading	!! EMERGENCY !! HR:115 T:39.2 A:2.8

6. OLED Display

6.1 Configuration

The SSD1306 128x64 OLED display was configured on I2C bus 1 at address 0x3C. The display was enabled via raspi-config (Interface Options → I2C) and verified using i2cdetect -y 1.

6.2 Display Screens

Screen	Content	Trigger
Sensor View	HR, Temperature, Acceleration, Timestamp	Normal operation (alternates every 2s)
Status View	Date, Time, ThingsBoard connection status	Normal operation (alternates every 2s)
Emergency View	!! EMERGENCY !!, all sensor values, timestamp	Any threshold exceeded
Startup Screen	Smartband V001, HSHL Project, Starting...	System boot
Shutdown Screen	System OFF, Goodbye!, HSHL Project	Keyboard interrupt

7. ThingsBoard Cloud Integration

All sensor data is published to the ThingsBoard EU cloud platform via MQTT on every 2-second cycle. The connection details are:

Property	Value
Host	mqtt.eu.thingsboard.cloud
Port	1883
Protocol	MQTT (paho-mqtt, CallbackAPIVersion.VERSION2)
Topic	v1/devices/me/telemetry
Payload Fields	heartRate, temperature, acceleration, status, hr_limit, temp_limit, acc_limit

The system operates in offline mode gracefully if the MQTT connection fails, continuing all local operations (OLED, LED, buzzer, CSV, Bluetooth) without interruption.

8. Real-Time Dashboard

A separate real-time graph dashboard (`smartband_plot.py`) was developed using `matplotlib`. It reads `sensor_data.csv` every second and renders three live charts:

- Heart Rate (BPM) — red line with threshold marker at 100 BPM
- Temperature (°C) — orange line with threshold marker at 38.5°C
- Acceleration (G) — blue line with threshold marker at 2.0G

The dashboard features a dark theme, a status banner that turns red during emergencies and green during normal operation, and live chart backgrounds that change color when thresholds are exceeded. The dashboard runs independently of the main system and can be started or stopped at any time.

9. Testing & Verification

Test	Method	Result
OLED Detection	<code>i2cdetect -y 1</code>	✓ 0x3C detected
OLED Display	<code>adafruit_ssd1306</code> text render	✓ Text displayed correctly
Pmod BT2 Power	Visual — LED blinking	✓ Slow blink = ready
Pmod BT2 UART	Python serial + \$\$\$ command	✓ CMD response received
UART Loopback	TXD0 → RXD0 jumper	✓ b'hello' echoed back
BT Pairing (Pi)	<code>bluetoothctl pair</code>	✓ Paired and trusted
BT Data Transfer	<code>rfcomm</code> + Python serial	✓ Data received on UART
Buzzer	<code>pinctrl GPIO17</code> toggle	✓ Audible buzz confirmed
LED Red	<code>pinctrl GPIO27</code> toggle	✓ Red illuminated
LED Green	<code>pinctrl GPIO26</code> toggle	✓ Green illuminated
Emergency Mode	Simulated spike (step % 8)	✓ LED flash + buzzer + OLED
ThingsBoard MQTT	<code>paho-mqtt</code> publish	✓ Data visible on dashboard
Android BT	Serial Bluetooth Terminal app	✓ Live data stream received
Live Graph	<code>smartband_plot.py</code>	✓ All 3 charts updating live

10. Issues Encountered & Resolutions

Issue	Root Cause	Resolution
OLED not detected on i2cdetect	I2C not enabled on Pi	Enabled via raspi-config → Interface Options → I2C
OLED no power (blank screen)	Wires in column j (not connected to extender)	Moved all wires to column e (directly on extender)
Pmod BT2 no response to \$\$\$	Pi 5 UART not mapped to GPIO14/15	Added enable_uart=1 to /boot/firmware/config.txt
Pmod BT2 CMD mode no response	Wrong serial device (/dev/serial0 vs /dev/ttyAMA0)	Used /dev/serial0 after enable_uart=1 reboot
BT pairing failed (passkey)	RN-42 uses fixed PIN not passkey	Set SA,0 (no auth) and SP,1234 (PIN) via UART commands
Phone could not connect	Pi rfcomm occupied the module	Cleared stored remote (SR,Z), switched to direct UART in v3
LED Green not lighting	Signal wire accidentally in GND row (a41)	Moved wire to correct LED Green row (a42)
font5x8.bin missing	File not in working directory	Downloaded from Adafruit GitHub to ~/smartband/

11. Next Steps

The following steps are planned for the next phase of the project:

- Assemble the Band01_Rev1 Smartband PCB hardware
- Activate the commented-out sensor reading calls in simple_peripheral.c and i2c_sensors.c firmware
- Replace simulated sensor values in smartband_full_v3.py with real BLE data received from the Smartband
- Implement BLE scanning on the Raspberry Pi to receive data directly from the Band01_Rev1 CC2640R2F chip
- Integrate the OledThread and AppThread tasks currently disabled in main.c
- Replace the 0x7788 placeholder value with real sensor readings in the BLE GATT profile

12. Conclusion

This phase successfully demonstrated a complete health monitoring pipeline on the Raspberry Pi 5. All hardware components were wired, tested, and integrated into a unified system. The Pmod BT2 Bluetooth module enables wireless data transmission to Android devices, the SSD1306 OLED provides a real-time local display, the RGB LED and buzzer provide visual and audible emergency alerts, and the ThingsBoard cloud platform provides remote monitoring capability.

The system is now ready for integration with the actual Smartband V001 hardware. The software architecture is modular and designed to easily replace simulated sensor data with

real readings from the Band01_Rev1 once the hardware is assembled and the firmware is activated.